Second Edition

# UNDERSTANDING DISTRIBUTED SYSTEMS

What every developer should know about large distributed applications




ROBERTO VITILLO

# Understanding Distributed Systems

Version 2.0.0

Roberto Vitillo

March 2022

# Contents

**Copyright** **ix**

**About the author** **xi**

**Acknowledgements** **xiii**

**Preface** **xv**

**1 Introduction** **1**
1.1 Communication . . . . . 2
1.2 Coordination . . . . . 3
1.3 Scalability . . . . . 3
1.4 Resiliency . . . . . 5
1.5 Maintainability . . . . . 6
1.6 Anatomy of a distributed system . . . . . 7

**I Communication** **11**

**2 Reliable links** **17**
2.1 Reliability . . . . . 18
2.2 Connection lifecycle . . . . . 18
2.3 Flow control . . . . . 20
2.4 Congestion control . . . . . 21
2.5 Custom protocols . . . . . 23

**3 Secure links** **25**
3.1 Encryption . . . . . 25
3.2 Authentication . . . . . 26
3.3 Integrity . . . . . 28
3.4 Handshake . . . . . 29

**4 Discovery** **31**

**5 APIs** **35**
5.1 HTTP . . . . . 37
5.2 Resources . . . . . 39
5.3 Request methods . . . . . 41
5.4 Response status codes . . . . . 42
5.5 OpenAPI . . . . . 43
5.6 Evolution . . . . . 45
5.7 Idempotency . . . . . 46

**II Coordination** **53**

**6 System models** **57**

**7 Failure detection** **61**

**8 Time** **63**
8.1 Physical clocks . . . . . 63
8.2 Logical clocks . . . . . 65
8.3 Vector clocks . . . . . 67

**9 Leader election** **71**
9.1 Raft leader election . . . . . 72
9.2 Practical considerations . . . . . 73

**10 Replication** **77**
10.1 State machine replication . . . . . 78
10.2 Consensus . . . . . 81
10.3 Consistency models . . . . . 83
10.4 Chain replication . . . . . 90

**11 Coordination avoidance** **95**
11.1 Broadcast protocols . . . . . 96
11.2 Conflict-free replicated data types . . . . . 98
11.3 Dynamo-style data stores . . . . . 103
11.4 The CALM theorem . . . . . 105
11.5 Causal consistency . . . . . 106
11.6 Practical considerations . . . . . 110

**12 Transactions** **111**
12.1 ACID . . . . . 112
12.2 Isolation . . . . . 113

12.3 Atomicity . . . . . 119
12.4 NewSQL . . . . . 122

**13 Asynchronous transactions** **127**
13.1 Outbox pattern . . . . . 128
13.2 Sagas . . . . . 130
13.3 Isolation . . . . . 133

**III Scalability** **137**

**14 HTTP caching** **145**
14.1 Reverse proxies . . . . . 148

**15 Content delivery networks** **151**
15.1 Overlay network . . . . . 151
15.2 Caching . . . . . 153

**16 Partitioning** **155**
16.1 Range partitioning . . . . . 157
16.2 Hash partitioning . . . . . 158

**17 File storage** **163**
17.1 Blob storage architecture . . . . . 163

**18 Network load balancing** **169**
18.1 DNS load balancing . . . . . 174
18.2 Transport layer load balancing . . . . . 175
18.3 Application layer load balancing . . . . . 178

**19 Data storage** **181**
19.1 Replication . . . . . 181
19.2 Partitioning . . . . . 184
19.3 NoSQL . . . . . 185

**20 Caching** **191**
20.1 Policies . . . . . 192
20.2 Local cache . . . . . 193
20.3 External cache . . . . . 194

**21 Microservices** **197**
21.1 Caveats . . . . . 199
21.2 API gateway . . . . . 202

**22 Control planes and data planes** **209**
22.1 Scale imbalance . . . . . 211
22.2 Control theory . . . . . 214

**23 Messaging** **217**
23.1 Guarantees . . . . . 221
23.2 Exactly-once processing . . . . . 223
23.3 Failures . . . . . 224
23.4 Backlogs . . . . . 224
23.5 Fault isolation . . . . . 225

**IV Resiliency** **229**

**24 Common failure causes** **233**
24.1 Hardware faults . . . . . 233
24.2 Incorrect error handling . . . . . 234
24.3 Configuration changes . . . . . 234
24.4 Single points of failure . . . . . 235
24.5 Network faults . . . . . 236
24.6 Resource leaks . . . . . 237
24.7 Load pressure . . . . . 238
24.8 Cascading failures . . . . . 238
24.9 Managing risk . . . . . 240

**25 Redundancy** **243**
25.1 Correlation . . . . . 244

**26 Fault isolation** **247**
26.1 Shuffle sharding . . . . . 248
26.2 Cellular architecture . . . . . 250

**27 Downstream resiliency** **253**
27.1 Timeout . . . . . 253
27.2 Retry . . . . . 255
27.3 Circuit breaker . . . . . 258

**28 Upstream resiliency** **261**
28.1 Load shedding . . . . . 261
28.2 Load leveling . . . . . 262
28.3 Rate-limiting . . . . . 263
28.4 Constant work . . . . . 269

**V Maintainability** **275**

**29 Testing** **279**
29.1 Scope . . . 280
29.2 Size . . . 281
29.3 Practical considerations . . . 283
29.4 Formal verification . . . 285

**30 Continuous delivery and deployment** **289**
30.1 Review and build . . . 290
30.2 Pre-production . . . 292
30.3 Production . . . 293
30.4 Rollbacks . . . 293

**31 Monitoring** **297**
31.1 Metrics . . . 298
31.2 Service-level indicators . . . 301
31.3 Service-level objectives . . . 304
31.4 Alerts . . . 306
31.5 Dashboards . . . 308
31.6 Being on call . . . 312

**32 Observability** **315**
32.1 Logs . . . 316
32.2 Traces . . . 319
32.3 Putting it all together . . . 321

**33 Manageability** **323**

**34 Final words** **327**

# Copyright

**Understanding Distributed Systems** by Roberto Vitillo

Copyright © Roberto Vitillo. All rights reserved.

The book's diagrams have been created with Excalidraw.

While the author has used good faith efforts to ensure that the information and instructions in this work are accurate, the author disclaims all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. The use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

# About the author

Authors generally write this page in the third person as if someone else is writing about them, but I like to do things a little bit differently.

I have over 10 years of experience in the tech industry as a software engineer, technical lead, and manager.

After getting my master's degree in computer science from the University of Pisa, I worked on scientific computing applications at the Berkeley Lab. The software I contributed is used to this day by the ATLAS experiment at the Large Hadron Collider.

Next, I worked at Mozilla, where I set the direction of the data platform from its very early days and built a large part of it, including the team.

In 2017, I joined Microsoft to work on an internal SaaS for telemetry. Since then, I have helped launch multiple public SaaS products, like Playfab and Customer Insights. The data ingestion platform I am responsible for is one of the largest in the world, ingesting millions of events per second from billions of devices worldwide.

# Acknowledgements

I am very thankful for the colleagues and mentors who inspired me and believed in me over the years. Thanks to Chiara Roda, Andrea Dotti, Paolo Calafiura, Vladan Djeric, Mark Reid, Pawel Chodarcewicz, and Nuno Cerqueira.

Rachael Churchill, Doug Warren, Vamis Xhagjika, Alessio Placitelli, Stefania Vitillo, and Alberto Sottile provided invaluable feedback on early drafts. Without them, the book wouldn't be what it is today.

A very special thanks to the readers of the first edition who reached out with feedback and suggestions over the past year and helped me improve the book: Matthew Williams, Anath B Chatterjee, Ryan Carney, Marco Vocialta, Curtis Washington, David Golden, Radosław Rusiniak, Sankaranarayanan Viswanathan, Praveen Barli, Abhijit Sarkar, Maki Pavlidis, Venkata Srinivas Namburi, Andrew Myers, Aaron Michaels, Jack Henschel, Ben Gummer, Luca Colantonio, Kofi Sarfo, Mirko Schiavone, and Gaurav Narula.

I am also very thankful for all the readers who left reviews or reached out to me and let me know they found the book useful. They gave me the motivation and energy to write a second edition.

Finally, and above all, thanks to my family: Rachell and Leonardo. Without your unwavering support, this book wouldn't exist.

# Preface

Learning to build distributed systems is hard, especially if they are large scale. It's not that there is a lack of information out there. You can find academic papers, engineering blogs, and even books on the subject. The problem is that the available information is spread out all over the place, and if you were to put it on a spectrum from theory to practice, you would find a lot of material at the two ends but not much in the middle.

That is why I decided to write a book that brings together the core theoretical and practical concepts of distributed systems so that you don't have to spend hours connecting the dots. This book will guide you through the fundamentals of large-scale distributed systems, with just enough details and external references to dive deeper. This is the guide I wished existed when I first started out.

If you are a developer working on the backend of web or mobile applications (or would like to be!), this book is for you. When building distributed applications, you need to be familiar with the network stack, data consistency models, scalability and reliability patterns, observability best practices, and much more. Although you can build applications without knowing much of that, you will end up spending hours debugging and re-architecting them, learning hard lessons that you could have acquired in a much faster and less painful way.

However, if you have several years of experience designing and building highly available and fault-tolerant applications that scale to millions of users, this book might not be for you. As an expert,

you are likely looking for depth rather than breadth, and this book focuses more on the latter since it would be impossible to cover the field otherwise.

The second edition is a complete rewrite of the previous edition. Every page of the first edition has been reviewed and where appropriate reworked, with new topics covered for the first time.

This book is available both in a physical and digital format. The digital version is updated occasionally, which is why the book has a version number. You can subscribe to receive updates from the book's landing page[1].

As no book is ever perfect, I'm always happy to receive feedback. So if you find an error, have an idea for improvement, or simply want to comment on something, always feel free to write me[2]. I love connecting with readers!

[1] https://understandingdistributed.systems/
[2] roberto@understandingdistributed.systems

# Chapter 1

# Introduction

> *"A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable."*
>
> – Leslie Lamport

Loosely speaking, a distributed system is a group of nodes that cooperate by exchanging messages over communication links to achieve some task. A node can generically refer to a physical machine, like a phone, or a software process, like a browser.

Why do we bother building distributed systems in the first place?

Some applications are inherently distributed. For example, the web is a distributed system you are very familiar with. You access it with a browser, which runs on your phone, tablet, desktop, or Xbox. Together with other billions of devices worldwide, it forms a distributed system.

Another reason for building distributed systems is that some applications require high availability and need to be resilient to single-node failures. For example, Dropbox replicates your data across multiple nodes so that the loss of a single one doesn't cause your data to be lost.

Some applications need to tackle workloads that are just too big to fit on a single node, no matter how powerful. For example, Google receives tens of thousands of search requests per second from all over the globe. There is no way a single node could handle that.

And finally, some applications have performance requirements that would be physically impossible to achieve with a single node. Netflix can seamlessly stream movies to your TV at high resolution because it has a data center close to you.

This book tackles the fundamental challenges that need to be solved to design, build, and operate distributed systems.

## 1.1 Communication

The first challenge derives from the need for nodes to communicate with each other over the network. For example, when your browser wants to load a website, it resolves the server's IP address from the URL and sends an HTTP request to it. In turn, the server returns a response with the page's content.

How are the request and response messages represented on the wire? What happens when there is a temporary network outage, or some faulty network switch flips a few bits in the messages? How does the server guarantee that no intermediary can snoop on the communication?

Although it would be convenient to assume that some networking library is going to abstract all communication concerns away, in practice, it's not that simple because abstractions leak[1], and you need to understand how the network stack works when that happens.

[1] "The Law of Leaky Abstractions," https://www.joelonsoftware.com/2002/11/11/the-law-of-leaky-abstractions/

## 1.2 Coordination

Another hard challenge of building distributed systems is that some form of coordination is required to make individual nodes work in unison towards a shared objective. This is particularly challenging to do in the presence of failures. The "two generals" problem is a famous thought experiment that showcases this.

Suppose two generals (nodes), each commanding their own army, need to agree on a time to jointly attack a city. There is some distance between the armies (network), and the only way to communicate is via messengers, who can be captured by the enemy (network failure). Under these assumptions, is there a way for the generals to agree on a time?

Well, general 1 could send a message with a proposed time to general 2. But since the messenger could be captured, general 1 wouldn't know whether the message was actually delivered. You could argue that general 2 could send a messenger with a response to confirm it received the original message. However, just like before, general 2 wouldn't know whether the response was actually delivered and another confirmation would be required. As it turns out, no matter how many rounds of confirmation are made, neither general can be certain that the other army will attack the city at the same time. As you can see, this problem is much harder to solve than it originally appeared.

Because coordination is such a key topic, the second part of the book is dedicated to understanding the fundamental distributed algorithms used to implement it.

## 1.3 Scalability

The performance of an application represents how efficiently it can handle *load*. Intuitively, load is anything that consumes the system's resources such as CPU, memory, and network bandwidth. Since the nature of load depends on the application's use cases and architecture, there are different ways to measure it. For example,

the number of concurrent users or the ratio of writes to reads are different forms of load.

For the type of applications discussed in this book, performance is generally measured in terms of throughput and response time. *Throughput* is the number of requests processed per second by the application, while *response time* is the time elapsed in seconds between sending a request to the application and receiving a response.

As load increases, the application will eventually reach its *capacity*, i.e., the maximum load it can withstand, when a resource is exhausted. The performance either plateaus or worsens at that point, as shown in Figure 1.1. If the load on the system continues to grow, it will eventually hit a point where most operations fail or time out.

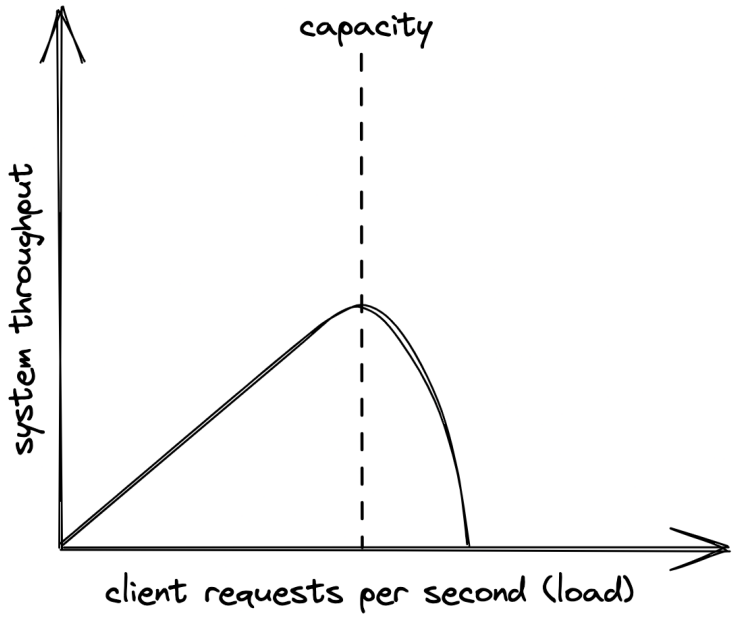



Figure 1.1: The system throughput on the y axis is the subset of client requests (x axis) that can be handled without errors and with low response times, also referred to as its goodput.

The capacity of a distributed system depends on its architecture, its implementation, and an intricate web of physical limitations like the nodes' memory size and clock cycle and the bandwidth and latency of network links. For an application to be scalable, a load increase should not degrade the application's performance. This requires increasing the capacity of the application at will.

A quick and easy way is to buy more expensive hardware with better performance, which is also referred to as *scaling up*. Unfortunately, this approach is bound to hit a brick wall sooner or later when such hardware just doesn't exist. The alternative is *scaling out* by adding more commodity machines to the system and having them work together.

Although procuring additional machines at will may have been daunting a few decades ago, the rise of cloud providers has made that trivial. In 2006 Amazon launched Amazon Web Services (AWS), which included the ability to rent virtual machines with its Elastic Compute Cloud (EC2[2]) service. Since then, the number of cloud providers and cloud services has only grown, democratizing the ability to create scalable applications.

In Part III of this book, we will explore the core architectural patterns and building blocks of scalable cloud-native applications.

## 1.4 Resiliency

A distributed system is resilient when it can continue to do its job even when failures happen. And at scale, anything that can go wrong will go wrong. Every component has a probability of failing — nodes can crash, network links can be severed, etc. No matter how small that probability is, the more components there are and the more operations the system performs, the higher the number of failures will be. And it gets worse because a failure of one component can increase the probability that another one will fail if the components are not well isolated.

[2] "Amazon EC2," https://aws.amazon.com/ec2/

Failures that are left unchecked can impact the system's *availability*[3], i.e., the percentage of time the system is available for use. It's a ratio defined as the amount of time the application can serve requests (*uptime*) divided by the total time measured (*uptime* plus *downtime*, i.e., the time the application can't serve requests).

Availability is often described with nines, a shorthand way of expressing percentages of availability. Three nines are typically considered acceptable by users, and anything above four is considered to be highly available.

| Availability % | Downtime per day |
|---|---|
| 90% ("one nine") | 2.40 hours |
| 99% ("two nines") | 14.40 minutes |
| 99.9% ("three nines") | 1.44 minutes |
| 99.99% ("four nines") | 8.64 seconds |
| 99.999% ("five nines") | 864 milliseconds |

If the system isn't resilient to failures, its availability will inevitably drop. Because of that, a distributed system needs to embrace failures and be prepared to withstand them using techniques such as redundancy, fault isolation, and self-healing mechanisms, which we will discuss in Part IV, *Resiliency*.

## 1.5 Maintainability

It's a well-known fact that the majority of the cost of software is spent after its initial development in maintenance activities, such as fixing bugs, adding new features, and operating it. Thus, we should aspire to make our systems easy to modify, extend and operate so that they are easy to maintain.

Any change is a potential incident waiting to happen. Good testing — in the form of unit, integration, and end-to-end tests — is a

[3] "AWS Well-Architected Framework, Availability," https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/availability.html

minimum requirement to modify or extend a system without worrying it will break. And once a change has been merged into the codebase, it needs to be released to production safely without affecting the system's availability.

Also, operators need to monitor the system's health, investigate degradations and restore the service when it can't self-heal. This requires altering the system's behavior without code changes, e.g., toggling a feature flag or scaling out a service with a configuration change.

Historically, developers, testers, and operators were part of different teams, but the rise of microservices and DevOps has changed that. Nowadays, the same team that designs and implements a system is also responsible for testing and operating it. That's a good thing since there is no better way to discover where a system falls short than being on call for it. Part V will explore best practices for testing and operating distributed systems.

## 1.6 Anatomy of a distributed system

Distributed systems come in all shapes and sizes. In this book, we are mainly concerned with backend applications that run on commodity machines and implement some kind of business service. So you could say a distributed system is a group of machines that communicate over network links. However, from a run-time point of view, a distributed system is a group of software processes that communicate via *inter-process communication* (IPC) mechanisms like HTTP. And from an implementation perspective, a distributed system is a group of loosely-coupled components (services) that communicate via APIs. All these are valid and useful architectural points of view. In the rest of the book, we will switch between them depending on which one is more appropriate to discuss a particular topic.

A *service* implements one specific part of the overall system's capabilities. At the core of a service sits the business logic, which exposes interfaces to communicate with the outside world. Some

interfaces define the operations that the service offers to its users. In contrast, others define the operations that the service can invoke on other services, like data stores, message brokers, etc.

Since processes can't call each other's interfaces directly, *adapters* are needed to connect IPC mechanisms to service interfaces. An inbound adapter is part of the service's *Application Programming Interface* (API); it handles the requests received from an IPC mechanism, like HTTP, by invoking operations defined in the service interfaces. In contrast, outbound adapters grant the business logic access to external services, like data stores. This architectural style is also referred to as the ports and adapters architecture[4]. The idea is that the business logic doesn't depend on technical details; instead, the technical details depend on the business logic (dependency inversion principle[5]). This concept is illustrated in Figure 1.2.

Going forward, we will refer to a process running a service as a *server*, and a process sending requests to a server as a *client*. Sometimes, a process will be both a client and a server. For simplicity, we will assume that an individual instance of a service runs entirely within a single server process. Similarly, we will also assume that a process has a single thread. These assumptions will allow us to neglect some implementation details that would only complicate the discussion without adding much value.

[4] "Ports And Adapters Architecture," http://wiki.c2.com/?PortsAndAdaptersArchitecture

[5] "Dependency inversion principle," https://en.wikipedia.org/wiki/Dependency_inversion_principle

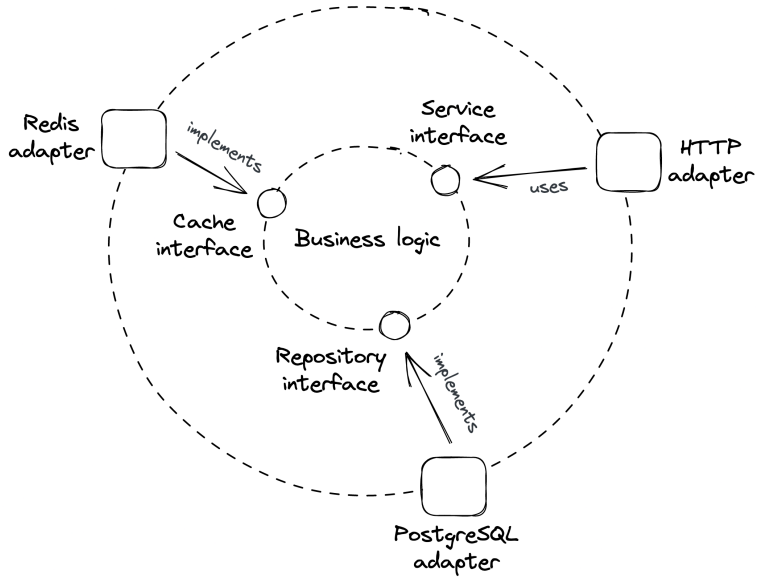



Figure 1.2: In this example, the business logic uses the repository interface, implemented by the PostgreSQL adapter, to access the database. In contrast, the HTTP adapter handles incoming requests by calling operations defined in the service interface.

# Part I

# Communication

# Introduction

*"The network is reliable."*

– Fallacies of distributed computing, L. Peter Deutsch

Communication between processes over the network, or *interprocess communication* (IPC), is at the heart of distributed systems — it's what makes distributed systems distributed. In order for processes to communicate, they need to agree on a set of rules that determine how data is processed and formatted. Network protocols specify such rules.

The protocols are arranged in a stack[6], where each layer builds on the abstraction provided by the layer below, and lower layers are closer to the hardware. When a process sends data to another through the network stack, the data moves from the top layer to the bottom one and vice-versa at the other end, as shown in Figure 1.3:

- The *link layer* consists of network protocols that operate on local network links, like Ethernet or Wi-Fi, and provides an interface to the underlying network hardware. Switches operate at this layer and forward Ethernet packets based on their destination MAC address[7].
- The *internet layer* routes packets from one machine to another across the network. The Internet Protocol (IP) is the core protocol of this layer, which delivers packets on a best-effort ba-

[6] "Internet protocol suite," https://en.wikipedia.org/wiki/Internet_protocol_suite

[7] "MAC address," https://en.wikipedia.org/wiki/MAC_address

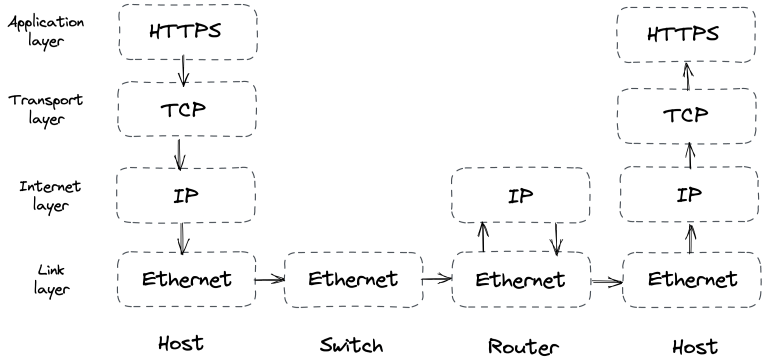



Figure 1.3: Internet protocol suite

sis (i.e., packets can be dropped, duplicated, or corrupted). Routers operate at this layer and forward IP packets to the next router along the path to their final destination.

- The *transport layer* transmits data between two processes. To enable multiple processes hosted on the same machine to communicate at the same time, port numbers are used to address the processes on either end. The most important protocol in this layer is the Transmission Control Protocol (TCP), which creates a reliable communication channel on top of IP.
- Finally, the *application layer* defines high-level communication protocols, like HTTP or DNS. Typically your applications will target this level of abstraction.

Even though each protocol builds on top of another, sometimes the abstractions leak. If you don't have a good grasp of how the lower layers work, you will have a hard time troubleshooting networking issues that will inevitably arise. More importantly, having an appreciation of the complexity of what happens when you make a network call will make you a better systems builder.

Chapter 2 describes how to build a reliable communication channel (TCP) on top of an unreliable one (IP), which can drop or duplicate data or deliver it out of order. Building reliable abstractions on top of unreliable ones is a common pattern we will encounter again in the rest of the book.

Chapter 3 describes how to build a secure channel (TLS) on top of a reliable one (TCP). Security is a core concern of any system, and in this chapter, we will get a taste of what it takes to secure a network connection from prying eyes and malicious agents.

Chapter 4 dives into how the phone book of the internet (DNS) works, which allows nodes to discover others using names. At its heart, DNS is a distributed, hierarchical, and eventually consistent key-value store. By studying it, we will get the first taste of eventual consistency[8] and the challenges it introduces.

Chapter 5 concludes this part by discussing how loosely coupled services communicate with each other through APIs by describing the implementation of a RESTful HTTP API built upon the protocols introduced earlier.

[8] We will learn more about consistency models in chapter 10.

# Chapter 2

# Reliable links

At the internet layer, the communication between two nodes happens by routing packets to their destination from one router to the next. Two ingredients are required for this: a way to address nodes and a mechanism to route packets across routers.

Addressing is handled by the IP protocol. For example, IPv6 provides a 128-bit address space, allowing $2^{128}$ addresses. To decide where to send a packet, a router needs to consult a local routing table. The table maps a destination address to the address of the next router along the path to that destination. The responsibility of building and communicating the routing tables across routers lies with the Border Gateway Protocol (BGP[1]).

Now, IP doesn't guarantee that data sent over the internet will arrive at its destination. For example, if a router becomes overloaded, it might start dropping packets. This is where TCP[2] comes in, a transport-layer protocol that exposes a reliable communication channel between two processes on top of IP. TCP guarantees that a stream of bytes arrives in order without gaps, duplication, or corruption. TCP also implements a set of stability patterns to

[1] "RFC 4271: A Border Gateway Protocol 4 (BGP-4)," https://datatracker.ietf.org/doc/html/rfc4271

[2] "RFC 793: Transmission Control Protocol," https://tools.ietf.org/html/rfc793

avoid overwhelming the network and the receiver.

## 2.1 Reliability

To create the illusion of a reliable channel, TCP partitions a byte stream into discrete packets called segments. The segments are sequentially numbered, which allows the receiver to detect holes and duplicates. Every segment sent needs to be acknowledged by the receiver. When that doesn't happen, a timer fires on the sending side and the segment is retransmitted. To ensure that the data hasn't been corrupted in transit, the receiver uses a checksum to verify the integrity of a delivered segment.

## 2.2 Connection lifecycle

A connection needs to be opened before any data can be transmitted on a TCP channel. The operating system manages the connection state on both ends through a *socket*. The socket keeps track of the state changes of the connection during its lifetime. At a high level, there are three states the connection can be in:

- The opening state in which the connection is being created.
- The established state in which the connection is open and data is being transferred.
- The closing state in which the connection is being closed.

In reality, this is a simplification, as there are more states[3] than the three above.

A server must be listening for connection requests from clients before a connection is established. TCP uses a three-way handshake to create a new connection, as shown in Figure 2.1:

1. The sender picks a random sequence number $x$ and sends a SYN segment to the receiver.

[3] "TCP State Diagram," https://en.wikipedia.org/wiki/Transmission_Control_Protocol#/media/File:Tcp_state_diagram_fixed_new.svg

2. The receiver increments $x$, chooses a random sequence number $y$, and sends back a SYN/ACK segment.
3. The sender increments both sequence numbers and replies with an ACK segment and the first bytes of application data.

The sequence numbers are used by TCP to ensure the data is delivered in order and without holes.

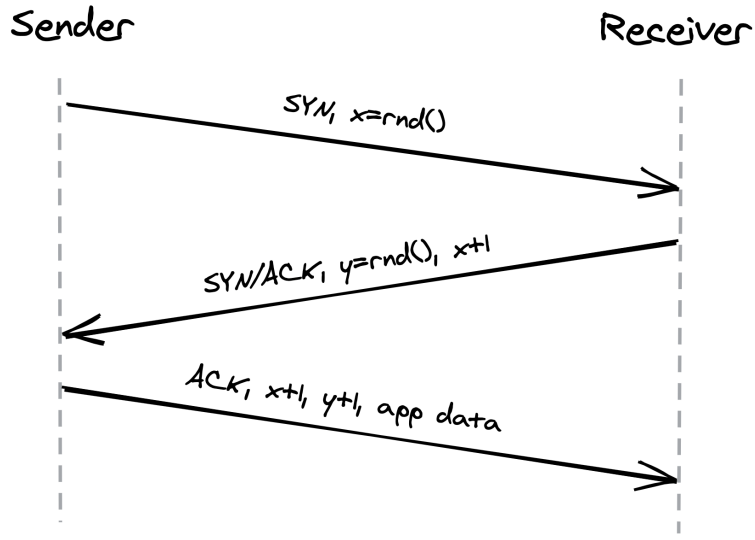



Figure 2.1: Three-way handshake

The handshake introduces a full round-trip in which no application data is sent. So until the connection has been opened, the bandwidth is essentially zero. The lower the round trip time is, the faster the connection can be established. Therefore, putting servers closer to the clients helps reduce this cold-start penalty.

After the data transmission is complete, the connection needs to be closed to release all resources on both ends. This termination phase involves multiple round-trips. If it's likely that another transmission will occur soon, it makes sense to keep the connection open to avoid paying the cold-start tax again.

Moreover, closing a socket doesn't dispose of it immediately as it

transitions to a waiting state (*TIME_WAIT*) that lasts several minutes and discards any segments received during the wait. The wait prevents delayed segments from a closed connection from being considered part of a new connection. But if many connections open and close quickly, the number of sockets in the waiting state will continue to increase until it reaches the maximum number of sockets that can be open, causing new connection attempts to fail. This is another reason why processes typically maintain connection pools to avoid recreating connections repeatedly.

## 2.3 Flow control

Flow control is a backoff mechanism that TCP implements to prevent the sender from overwhelming the receiver. The receiver stores incoming TCP segments waiting to be processed by the application into a receive buffer, as shown in Figure 2.2.

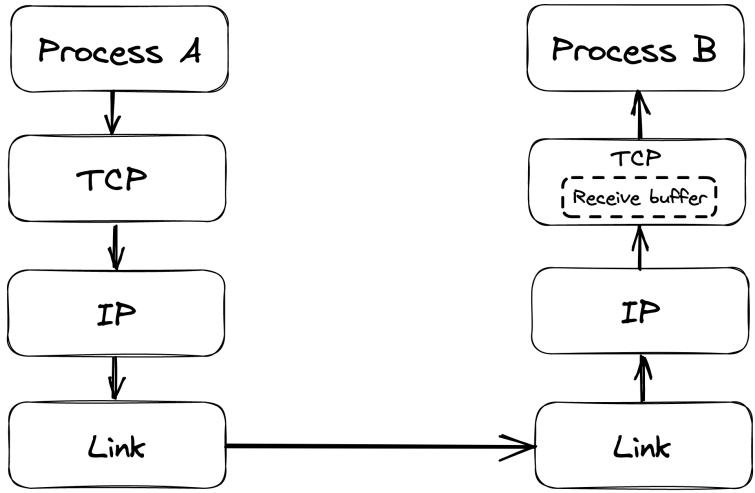



Figure 2.2: The receive buffer stores data that hasn't yet been processed by the destination process.

The receiver also communicates the size of the buffer to the sender whenever it acknowledges a segment, as shown in Figure 2.3.

Assuming it's respecting the protocol, the sender avoids sending more data than can fit in the receiver's buffer.

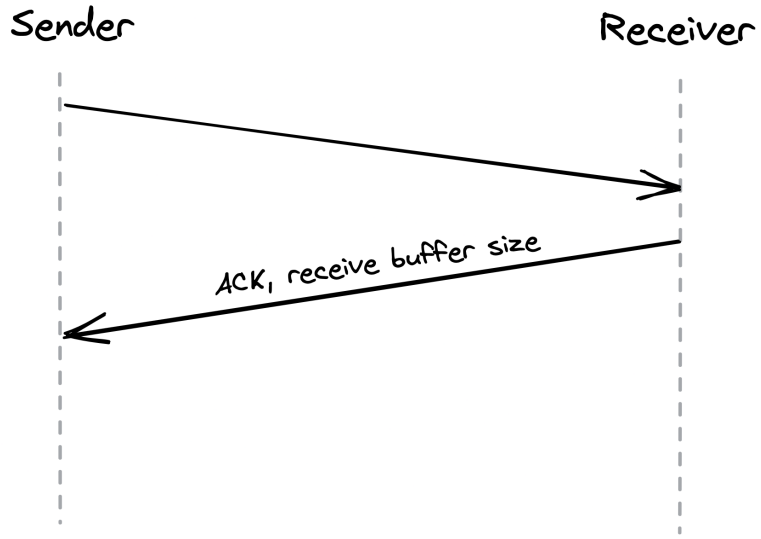



Figure 2.3: The size of the receive buffer is communicated in the headers of acknowledgment segments.

This mechanism is not too dissimilar to rate-limiting at the service level, a mechanism that rejects a request when a specific quota is exceeded (see section 28.3). But, rather than rate-limiting on an API key or IP address, TCP is rate-limiting on a connection level.

## 2.4 Congestion control

TCP guards not only against overwhelming the receiver, but also against flooding the underlying network. The sender maintains a so-called *congestion window*, which represents the total number of outstanding segments that can be sent without an acknowledgment from the other side. The smaller the congestion window is, the fewer bytes can be in flight at any given time, and the less bandwidth is utilized.

When a new connection is established, the size of the congestion window is set to a system default. Then, for every segment acknowledged, the window increases its size exponentially until it reaches an upper limit. This means we can't use the network's full capacity right after a connection is established. The shorter the round-trip time (RTT), the quicker the sender can start utilizing the underlying network's bandwidth, as shown in Figure 2.4.

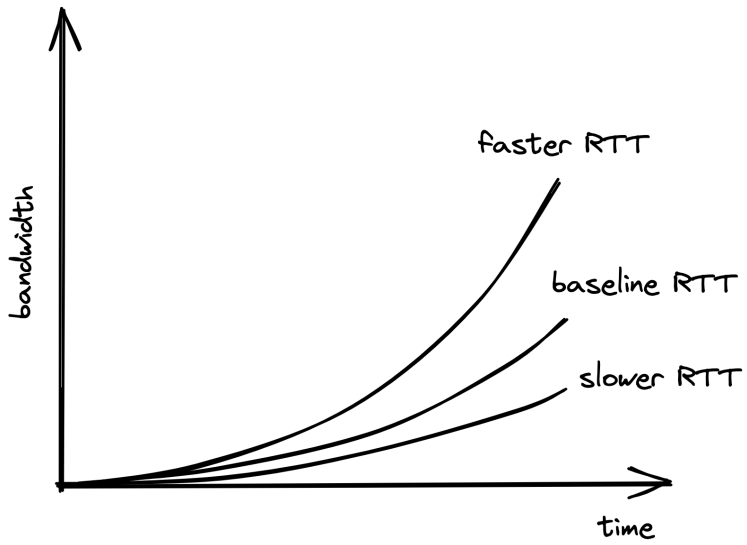



Figure 2.4: The shorter the RTT, the quicker the sender can start utilizing the underlying network's bandwidth.

What happens if a segment is lost? When the sender detects a missed acknowledgment through a timeout, a mechanism called *congestion avoidance* kicks in, and the congestion window size is reduced. From there onwards, the passing of time increases the window size[4] by a certain amount, and timeouts decrease it by another.

As mentioned earlier, the size of the congestion window defines the maximum number of bytes that can be sent without receiving

[4] "CUBIC: A New TCP-Friendly High-Speed TCP Variant," https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.153.3152&rep=rep1&type=pdf

an acknowledgment. Because the sender needs to wait for a full round trip to get an acknowledgment, we can derive the maximum theoretical bandwidth by dividing the size of the congestion window by the round trip time:

$$\text{Bandwidth} = \frac{\text{WinSize}}{\text{RTT}}$$

The equation[5] shows that bandwidth is a function of latency. TCP will try very hard to optimize the window size since it can't do anything about the round-trip time. However, that doesn't always yield the optimal configuration. Due to the way congestion control works, the shorter the round-trip time, the better the underlying network's bandwidth is utilized. This is more reason to put servers geographically close to the clients.

## 2.5 Custom protocols

TCP's reliability and stability come at the price of lower bandwidth and higher latencies than the underlying network can deliver. If we drop the stability and reliability mechanisms that TCP provides, what we get is a simple protocol named *User Datagram Protocol*[6] (UDP) — a connectionless transport layer protocol that can be used as an alternative to TCP.

Unlike TCP, UDP does not expose the abstraction of a byte stream to its clients. As a result, clients can only send discrete packets with a limited size called *datagrams*. UDP doesn't offer any reliability as datagrams don't have sequence numbers and are not acknowledged. UDP doesn't implement flow and congestion control either. Overall, UDP is a lean and bare-bones protocol. It's used to bootstrap custom protocols, which provide some, but not all, of the stability and reliability guarantees that TCP does[7].

[5] "Bandwidth-delay product," https://en.wikipedia.org/wiki/Bandwidth-delay_product

[6] "RFC 768: User Datagram Protocol," https://datatracker.ietf.org/doc/html/rfc768

[7] As we will later see, HTTP 3 is based on UDP to avoid some of TCP's shortcomings.

For example, in multiplayer games, clients sample gamepad events several times per second and send them to a server that keeps track of the global game state. Similarly, the server samples the game state several times per second and sends these snapshots back to the clients. If a snapshot is lost in transmission, there is no value in retransmitting it as the game evolves in real-time; by the time the retransmitted snapshot would get to the destination, it would be obsolete. This is a use case where UDP shines; in contrast, TCP would attempt to redeliver the missing data and degrade the game's experience.




GET THE REST OF THE BOOK AT
HTTPS://UNDERSTANDINGDISTRIBUTED.SYSTEMS/